**Quality control effectiveness.** A software team should apply limited resources in a way that has the highest likelihood of achieving a high-quality result. SQA analyzes the allocation of resources for reviews and testing to assess whether they are being allocated in the most effective manner.

Table 17.1 (adapted from [Hya96]) identifies the attributes that are indicators for the existence of quality for each of the goals discussed. Metrics that can be used to indicate the relative strength of an attribute are also shown.

**TABLE 17.1**

**Software quality goals, attributes, and metrics**

Source: Adapted from Hyatt, L., and Rosenberg, L., "A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality," NASA SATC, 1996.

| Goal | Attribute | Metric |
|---|---|---|
| **Requirement quality** | Ambiguity | Number of ambiguous modifiers (e.g., many, large, human-friendly) |
| | Completeness | Number of TBA, TBD |
| | Understandability | Number of sections/subsections |
| | Volatility | Number of changes per requirement |
| | | Time (by activity) when change is requested |
| | Traceability | Number of requirements not traceable to design/code |
| | Model clarity | Number of UML models |
| | | Number of descriptive pages per model |
| | | Number of UML errors |
| **Design quality** | Architectural integrity | Existence of architectural model |
| | Component completeness | Number of components that trace to architectural model |
| | | Complexity of procedural design |
| | Interface complexity | Average number of pick to get to a typical function or content |
| | | Layout appropriateness |
| | Patterns | Number of patterns used |
| **Code quality** | Complexity | Cyclomatic complexity |
| | Maintainability | Design factors |
| | Understandability | Percent internal comments |
| | | Variable naming conventions |
| | | Percent reused components |
| | Reusability | Percent reused component |
| | Documentation | Readability index |
| **QC effectiveness** | Resource allocation | Staff hour percentage per activity |
| | Completion rate | Actual vs. budgeted completion time |
| | Review effectiveness | See review metrics |
| | Testing effectiveness | Number of errors found and criticality |
| | | Effort required to correct an error |
| | | Origin of error |

## 17.5 Formal Approaches to SQA

In the preceding sections, we have argued that software quality is everyone's job and that it can be achieved through competent software engineering practice as well as through the application of technical reviews, a multitiered testing strategy, better control of software work products and the changes made to them, and the application of accepted software engineering standards and process frameworks. In addition, quality can be defined in terms of a broad array of quality attributes and measured (indirectly) using a variety of indices and metrics.

Over the past three decades, a small, but vocal, segment of the software engineering community has argued that a more formal approach to software quality assurance is required. It can be argued that a computer program is a mathematical object. A rigorous syntax and semantics can be defined for every programming language, and a rigorous approach to the specification of software requirements is available. If the requirements model (specification) and the programming language can be represented in a rigorous manner, it should be possible to apply mathematic proof of correctness to demonstrate that a program conforms exactly to its specifications.

Attempts to prove programs correct are not new. Dijkstra [Dij76a] and Linger, Mills, and Witt [Lin79], among others, advocated proofs of program correctness and tied these to the use of structured programming concepts. Although formal methods are interesting to some software engineering researchers, commercial developers rarely make use of formal methods in 2018.

## 17.6 Statistical Software Quality Assurance

Statistical quality assurance reflects a growing trend throughout the industry to become more quantitative about quality. For software, statistical quality assurance implies the following steps:

1. Information about software errors and defects is collected and categorized.
2. An attempt is made to trace each error and defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the *vital few*).
4. Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

This relatively simple concept represents an important step toward the creation of an adaptive software process in which changes are made to improve those elements of the process that introduce error.

### 17.6.1 A Generic Example

To illustrate the use of statistical methods for software engineering work, assume that a software engineering organization collects information on errors and defects for a period of 1 year. Some of the errors are uncovered as software is being developed. Other defects are encountered after the software has been released to its end users.

Although hundreds of different problems are uncovered, all can be tracked to one (or more) of the following causes:

- Incomplete or erroneous specifications (IES)
- Misinterpretation of customer communication (MCC)
- Intentional deviation from specifications (IDS)
- Violation of programming standards (VPS)
- Error in data representation (EDR)
- Inconsistent component interface (ICI)
- Error in design logic (EDL)
- Incomplete or erroneous testing (IET)
- Inaccurate or incomplete documentation (IID)
- Error in programming language translation of design (PLT)
- Ambiguous or inconsistent human/computer interface (HCI)
- Miscellaneous (MIS)

To apply statistical SQA, a table is built (see Table 17.2). The table indicates that IES, MCC, and EDR are the vital few causes that account for 53 percent of all errors. It should be noted, however, that IES, EDR, PLT, and EDL would be selected as the vital few causes if only serious errors are considered. Once the vital few causes are determined, the software engineering organization can begin corrective action. For example, to correct MCC, you might implement requirements gathering techniques (Chapter 7) to improve the quality of customer communication and specifications. To improve EDR, you might acquire tools for data modeling and perform more stringent data design reviews.

**TABLE 17.2**

**Data collection for statistical SQA**

| | Total | | Serious | | Moderate | | Minor | |
|---|---|---|---|---|---|---|---|---|
| **Error** | **No.** | **%** | **No.** | **%** | **No.** | **%** | **No.** | **%** |
| IES | 205 | 22% | 34 | 27% | 68 | 18% | 103 | 24% |
| MCC | 156 | 17% | 12 | 9% | 68 | 18% | 76 | 17% |
| IDS | 48 | 5% | 1 | 1% | 24 | 6% | 23 | 5% |
| VPS | 25 | 3% | 0 | 0% | 15 | 4% | 10 | 2% |
| EDR | 130 | 14% | 26 | 20% | 68 | 18% | 36 | 8% |
| ICI | 58 | 6% | 9 | 7% | 18 | 5% | 31 | 7% |
| EDL | 45 | 5% | 14 | 11% | 12 | 3% | 19 | 4% |
| IET | 95 | 10% | 12 | 9% | 35 | 9% | 48 | 11% |
| IID | 36 | 4% | 2 | 2% | 20 | 5% | 14 | 3% |
| PLT | 60 | 6% | 15 | 12% | 19 | 5% | 26 | 6% |
| HCI | 28 | 3% | 3 | 2% | 17 | 4% | 8 | 2% |
| MIS | 56 | 6% | 0 | 0% | 15 | 4% | 41 | 9% |
| Totals | 942 | 100% | 128 | 100% | 379 | 100% | 435 | 100% |

It is important to note that corrective action focuses primarily on the vital few. As the vital few causes are corrected, new candidates pop to the top of the stack.

Statistical quality assurance techniques for software have been shown to provide substantial quality improvement (e.g., [Rya11], [Art97]). In some cases, software organizations have achieved a 50 percent reduction per year in defects after applying these techniques.

The application of the statistical SQA and the Pareto principle can be summarized in a single sentence: *Spend your time focusing on things that really matter, but first be sure that you understand what really matters!*

### 17.6.2 Six Sigma for Software Engineering

Six Sigma is the most widely used strategy for statistical quality assurance in industry today. Originally popularized by Motorola in the 1980s, the Six Sigma strategy "is a business-management strategy designed to improve the quality of process outputs by minimizing variation and causes of defects in processes. It is a subset of the Total Quality Management (TQM) methodology with a heavy focus on statistical applications used to reduce costs and improve quality" [Voe18]. The term *Six Sigma* is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high-quality standard. The Six Sigma methodology defines three core steps:

- *Define* customer requirements and deliverables and project goals via well-defined methods of customer communication.
- *Measure* the existing process and its output to determine current quality performance (collect defect metrics).
- *Analyze* defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps:

- *Improve* the process by eliminating the root causes of defects.
- *Control* the process to ensure that future work does not reintroduce the causes of defects.

These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

If an organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

- *Design* the process to (1) avoid the root causes of defects and (2) to meet customer requirements.
- *Verify* that the process model will, in fact, avoid defects and meet customer requirements.

This variation is sometimes called the DMADV (define, measure, analyze, design, and verify) method.

A comprehensive discussion of Six Sigma is best left to resources dedicated to the subject. If you have further interest, see [Voe18], [Pyz14], and [Sne18].

## 17.7 SOFTWARE RELIABILITY

There is no doubt that the reliability of a computer program is an important element of its overall quality. If a program repeatedly and frequently fails to perform, it matters little whether other software quality factors are acceptable.

Software reliability, unlike many other quality factors, can be measured directly and estimated using historical and developmental data. *Software reliability* is defined in statistical terms as "the probability of failure-free operation of a computer program in a specified environment for a specified time" [Mus87]. To illustrate, program *X* is estimated to have a reliability of 0.999 over eight elapsed processing hours. In other words, if program *X* were to be executed 1000 times and require a total of 8 hours of elapsed processing time (execution time), it is likely to operate correctly (without failure) 999 times.

Whenever software reliability is discussed, a pivotal question arises: What is meant by the term *failure*? In the context of any discussion of software quality and reliability, failure is nonconformance to software requirements. Yet, even within this definition, there are gradations. Failures can be only annoying or catastrophic. One failure can be corrected within seconds, while another requires weeks or even months to correct. Complicating the issue even further, the correction of one failure may in fact result in the introduction of other errors that ultimately result in other failures.

### 17.7.1 Measures of Reliability and Availability

Early work in software reliability attempted to extrapolate the mathematics of hardware reliability theory to the prediction of software reliability. Most hardware-related reliability models are predicated on failure due to wear rather than failure due to design defects. In hardware, failures due to physical wear (e.g., the effects of temperature, corrosion, shock) are more likely than a design-related failure. Unfortunately, the opposite is true for software. In fact, all software failures can be traced to design or implementation problems; wear (see Chapter 1) is not a factor.

There has been an ongoing debate over the relationship between key concepts in hardware reliability and their applicability to software. Although an irrefutable link has yet to be established, it is worthwhile to consider a few simple concepts that apply to both system elements.

If we consider a computer-based system, a simple measure of reliability is *mean time between failure* (MTBF):[3]

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

where the acronyms MTTF and MTTR are *mean time to failure* and *mean time to repair*,[4] respectively.

3 It is important to note that MTBF and related measures are based on CPU time, not wall clock time.

4 Although debugging (and related corrections) may be required following a failure, in many cases the software will work properly after a restart with no other change.

Many researchers argue that MTBF is a far more useful measure than other quality-related software metrics discussed in Chapter 23. Stated simply, an end user is concerned with failures, not with the total defect count. Because each defect contained within a program does not have the same failure rate, the total defect count provides little indication of the reliability of a system. For example, consider a program that has been in operation for 3000 processor hours without failure. Many defects in this program may remain undetected for tens of thousands of hours before they are discovered. The MTBF of such obscure errors might be 30,000 or even 60,000 processor hours. Other defects, not yet discovered, might have a failure rate of 4000 or 5000 hours. Even if every one of the first category of errors (those with long MTBF) is removed, the impact on software reliability is negligible.

However, MTBF can be problematic for two reasons: (1) it projects a time span between failures but does not provide us with a projected failure rate, and (2) MTBF can be misinterpreted to mean average life span even though this is *not* what it implies.

An alternative measure of reliability is *failures in time* (FIT)—a statistical measure of how many failures a component will have over 1 billion hours of operation. Therefore, 1 FIT is equivalent to one failure in every billion hours of operation.

In addition to a reliability measure, you should also develop a measure of availability. *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software. Of course, some aspects of availability have nothing to do with failure. For example, scheduling downtime (for support functions) causes the software to be unavailable. For a comprehensive discussion of software reliability measures, see [Laz11].

### 17.7.2 Use of AI to Model Reliability

Some software engineers view data science as the application of artificial intelligence techniques to solve software engineering problems. One of the things artificial intelligence methods attempt to do is provide reasonable solutions to problems where the needed data may be incomplete. *Software reliability* is defined as the probability of failure-free software operation for a specified time period in a specified environment. This means that we can never know the exact moment when a software product will fail because we will never have the complete data needed to calculate the probability.

Software engineers have used statistical techniques based on Bayes' theorem[5] in quantitative decision-making situations for several years. *Bayesian inference* is a method of statistical inference in which Bayes' theorem is used to update the probability

5 Bayes' theorem for conditional probabilities is P(A|B) = (P(B|A) * P(A)) / P(B). For more details, see http://www.statisticshowto.com/bayes-theorem-problems/.